

Project 1 Report

FIR Filter Design

Anthony Betancourt

David Shaowen Wang

WES 237C: Oct 08, 2026

Overview

Project 1 introduces the Vitis HLS design flow by implementing a finite impulse response (FIR) filter in C/C++ and synthesizing it to RTL for the Pynq-Z2 board. The work starts with a functionally correct 11-tap filter (FIR11), scales to a 128-tap filter (FIR128) that is used for a guided design space exploration of throughput, latency, and area, and ends with the FIR11 design running on the Zynq FPGA through the PYNQ infrastructure.

At a high level, the project required us to:

- Implement FIR11 and FIR128 designs whose output matches the golden output (out.gold.dat)
- Use arbitrary-precision types (ap_int<>) to trade bitwidth against resource usage
- Apply HLS optimizations: pipelining, removing conditionals, loop partitioning, loop unrolling, and array partitioning
- Combine optimizations into a highest-throughput FIR128 architecture
- Report latency, initiation interval, resource usage, and throughput for every design

Links to Github/Video:

The code for every design in this report is available at the following URLs:

1. David's Github Repo up to last question :
https://github.com/swwang2000-creator/WES237C_FA26
Note: the report for each run has been saved inside reports folder (some of them are missing)
2. Anthony's github for last question: <https://github.com/aebetancourt14/wes237C-HES.git>

Design Environment

Table 1: Tools and settings used for every design

Item	Setting
HLS tool	Vitis HLS 2024.2
Target part	xc7z020c1g400-1 (Pynq-Z2)
Target clock	10 ns (100 MHz)
Top function	void fir(data_t *y, data_t x)
C simulation	make test: runs fir_test.cpp on input.dat (chirp) and diffs out.dat against out.gold.dat

<u>Item</u>	<u>Setting</u>
C synthesis	make hls, then make report to copy fir_csynth.rpt

Top-Down Methodology

The filter computes one output sample from the N most recent input samples:

$$y[n] = \sum c[k] \cdot x[n - k], \quad k = 0 \dots N - 1$$

The top-level function `fir()` is called once per input sample. The testbench reads each sample of the chirp, calls `fir()`, and writes the result to `out.dat`. Inside `fir()`, a static array `shift_reg[N]` holds the delay line so that it persists between calls, the constant array `c[N]` holds the coefficients, and the accumulator `acc` forms the sum of products. Vitis HLS maps the delay line to registers or a memory, each multiply and add to an operator, and the loop to a state machine. The optimizations in Questions 2 to 7 change how many of those operators exist and how many clock cycles each sample takes.

Measurement Method

All results are the top-level numbers in `fir_csynth.rpt`. Latency and Interval come from “Latency: Summary”, using the maximum when the min/max differ. Resource usage is the “Used” (total) line of “Utilization Estimates: Summary”. Throughput is calculated from the Estimated Clock Period in the report, not from the 10 ns target:

$$\text{Throughput (MHz)} = 1000 / (\text{Estimated clock period (ns)} \times \text{clock cycles per sample})$$

where clock cycles per sample is the top-level Interval, the number of cycles between successive calls of `fir()`. Throughput is stated in kHz or MHz, whichever is the more readable unit for that design.

Results

Question 1 – FIR11 Baseline

The 11-tap filter uses a single loop that shifts the delay line and accumulates the products in the same pass, running from the oldest tap to the newest so that no sample is overwritten before it is moved. No pragmas or other optimizations were applied.

Q1.(a) and Q1.(b) Latency and initiation interval (in clock cycles) and the BRAM, DSP, FF and LUT usage of the FIR11 baseline are listed in the table below.

Table 2: FIR11 baseline: performance and resource usage

<u>Design</u>	<u>Latency (cycles)</u>	<u>Interval (cycles)</u>	<u>Loop II</u>	<u>Est. clock (ns)</u>	<u>Throughput</u>	<u>BRAM</u>	<u>DSP</u>	<u>FF</u>	<u>LUT</u>
FIR11 baseline (no pragmas)	19	20	1	6.912	7.23 MHz	0	2	733	383

At the function level, latency is 19 cycles and the interval is 20 cycles. Calls run one after another, since there is no function-level pipeline. With the estimated clock period of 6.912 ns, throughput is $1000 / (6.912 \times 20) = 7.23$ MHz.

At the loop level, Vitis HLS automatically pipelined `Shift_Accum_Loop` with `II = 1`, although no pragma was written. The loop latency is 16 cycles and the loop interval is 11 cycles ($11 \text{ iterations} \times II = 1$); the extra 5 cycles are the pipeline draining (for the `i == 0` case). The loop already reaches the ideal `II` of 1, so the throughput bottleneck is the serial function interval of 20 cycles, not the loop.

FIR128 Baseline Implementation

FIR128 uses the same single-loop structure with `N = 128`, `int` for all three types, and no pragmas. This design (folder `Baseline`) is the reference point for Questions 2 to 7. Its numbers are reported in Q3.(a).

Question 2 – Variable Bitwidths

`coef_t` and `acc_t` were changed in `fir.h` from `int` to `ap_int<W>` (with `#include "ap_int.h"`). Every combination below was checked against the golden output before synthesis.

Q2.(a) Three bitwidth combinations, including the 32-bit baseline:

Table 3: FIR128 with different `coef_t` / `acc_t` bitwidths

Bitwidths (bits)	Latency (cycles)	Interval (cycles)	Loop II	Est. clock (ns)	Throughput	BRAM	DSP	FF	LUT
Baseline: <code>coef_t int</code> , <code>acc_t int</code> (32 bits)	135	136	1	6.912	1.06 MHz	2	2	588	315
<code>coef_t ap_int<8></code> , <code>acc_t ap_int<24></code>	134	135	1	6.712	1.10 MHz	2	1	324	287
<code>coef_t ap_int<5></code> , <code>acc_t ap_int<16></code>	134	135	1	6.712	1.10 MHz	2	1	244	239

Narrowing the types left the cycle counts essentially unchanged (latency $135 \rightarrow 134$ cycles) and BRAM at 2, but it removed one DSP block ($2 \rightarrow 1$) and reduced the logic: FF fell from 588 to 324 and then to 244, and LUT from 315 to 287 and then to 239.

Q2.(b) `coef_t`: 5 bits. The FIR128 coefficients range from -11 to $+11$. A signed W -bit integer covers -2^{W-1} to $2^{W-1} - 1$, so `ap_int<4>` (-8 to 7) cannot hold them and `ap_int<5>` (-16 to 15) is the minimum.

`acc_t`: 16 bits. This was determined by experiment: `ap_int<16>` is the smallest accumulator width for which the output still matched the golden output (third row of Table 3).

Question 3 – Pipelining

The pipeline pragma was placed inside the body of the `for` loop, not on the function.

Q3.(a) and Q3.(b) Baseline FIR128 (no pragma) compared with automatic pipelining turned off:

Table 4: FIR128 baseline and non-pipelined design

Design	Latency (cycles)	Interval (cycles)	Loop II	Est. clock (ns)	Throughput	BRAM	DSP	FE	LUT
(a) Baseline, no pragma	135	136	1	6.912	1.06MHZ	2	2	588	315
(b) #pragma HLS pipeline off	641	642	n/a	6.912	0.23MHZ	2	2	412	279

Q3.(c) Manual pipelining with the loop II increased in steps of one cycle until throughput stopped changing:

Table 5: FIR128 with #pragma HLS pipeline II=<value>

Requested II	Latency (cycles)	Interval (cycles)	Loop II	Est. clock (ns)	Throughput	BRAM	DSP	FE	LUT
II = 1	135	136	1	6.912	1.06MHZ	3	2	610	305
II = 2	262	263	2	6.912	0.55Mhz	2	2	513	321
II = 3	389	390	3	6.912	0.37Mhz	2	2	480	318
II = 4	516	517	4	6.912	0.28MHZ	2	2	480	323
II = 5	643	644	5	6.912	0.22MHZ	2	2	455	300

Q3.(d) II = 6. One iteration of the loop takes at most 5 cycles (the non-pipelined report shows an iteration latency of 2 to 5 cycles: 2 for the `if` branch and 5 for the `else` branch). At II = 5 consecutive iterations therefore no longer overlap, and the latency of 643 cycles is essentially that of the non-pipelined design in Q3.(b) (641 cycles). Setting II to 6 or more does not make sense, because it only adds idle cycles.

Q3.(e) The default II is 1. With no pragma, Vitis HLS pipelines the loop automatically at II = 1, which is already the most aggressive setting: the baseline in Q3.(a) has the same latency and interval (135 and 136 cycles) as the II = 1 row of Table 5.

Question 4 – Removing Conditional Statements

The `if (i == 0)` branch was hoisted out of the loop. The loop now runs from $N - 1$ down to 1 and only shifts and accumulates. The newest sample is handled once after the loop (`acc += x * c[0]` and `shift_reg[0] = x`).

Q4.(a) Automatically pipelined design, with and without the conditional:

Table 6: Effect of the conditional on the automatically pipelined design

Design	Latency (cycles)	Interval (cycles)	Loop II	Est. clock (ns)	Throughput	BRAM	DSP	FE	LUT
With conditional (baseline)	135	136	1	6.912	1.06MHZ	2	2	588	315
Without conditional	134	135	1	6.912	1.07MHZ	2	2	418	316

Q4.(b) Non-pipelined design (#pragma HLS pipeline off), with and without the conditional:

Table 7: Effect of the conditional on the non-pipelined design

<u>Design</u>	<u>Latency (cycles)</u>	<u>Interval (cycles)</u>	<u>Loop II</u>	<u>Est. clock (ns)</u>	<u>Throughput</u>	<u>BRAM</u>	<u>DSP</u>	<u>FF</u>	<u>LUT</u>
With conditional (Q3.(b))	641	642	n/a	6.912	0.23Mhz	2	2	412	279
Without conditional	636	637	n/a	6.912	0.23Mhz	2	2	345	270

Removing the conditional barely changes the cycle counts or LUT usage, but it significantly reduces the number of flip-flops. In the automatically pipelined design, latency goes from 135 to 134 cycles and FF from 588 to 418 (about 29% fewer), with LUT unchanged (315 and 316). In the non-pipelined design, latency goes from 641 to 636 cycles, FF from 412 to 345 (about 16% fewer) and LUT from 279 to 270. Without the branch, the iteration latency is also no longer variable; it was 2 to 5 cycles depending on which branch ran.

Question 5 – Loop Partitioning

Q5.(a) Each iteration of the original loop does two independent jobs: it shifts the delay line (`shift_reg[i] = shift_reg[i-1]`) and it performs a multiply–accumulate (`acc += shift_reg[i] * c[i]`). Splitting them into a shift loop and a multiply–accumulate (MAC) loop, labelled `SECOND_LOOP` in the code, lets each loop be pipelined, unrolled, and given its own memory architecture separately. The shift loop has no arithmetic, and the MAC loop only reads the delay line. The newest sample is handled between the two loops.

Q5.(b) and Q5.(c) Design with and without loop partitioning, and the partitioned design with loop unrolling applied:

Table 8: Loop partitioning and loop unrolling

<u>Design</u>	<u>Latency (cycles)</u>	<u>Interval (cycles)</u>	<u>Loop II</u>	<u>Est. clock (ns)</u>	<u>Throughput</u>	<u>BRAM</u>	<u>DSP</u>	<u>FF</u>	<u>LUT</u>
(b) Single loop (Q4 design)	134	135	1	6.912	1.07MHZ	2	2	418	316
(b) Partitioned: shift loop + MAC loop	266	267	Shift 1 mac1	6.912	0.54MHZ	3	2	463	395
(c) Partitioned + unroll factor 4	203	204	2	7.206	0.68Mhz	6	4	999	813

Q5.(b) Loop partitioning on its own made the design about two times slower: latency went from 134 to 266 cycles, because the two loops now run one after the other. BRAM rose from 2 to 3, LUT from 316 to 395 and FF from 418 to 463.

Q5.(c) With the MAC loop unrolled by a factor of 4, latency is 203 cycles. Ideally it would be $134 + 134/4 \approx 167$ cycles, but the result is close to $134 + 134/2$: a 4× unroll gave only about a 2× speed-up of that loop, because each unrolled iteration takes 2 cycles (the MAC loop II is 2). Our first explanation was that one iteration now reads four array locations and so needs four

times the ports; Question 6 shows that the limit is the accumulation itself. BRAM doubled (3 → 6) to feed the four multipliers (DSP 2 → 4), and LUT and FF roughly doubled as well (395 → 813 and 463 → 999).

Q5.(d) Unrolling and pipelining are orthogonal: unrolling adds spatial parallelism (K copies of the loop body in hardware), and pipelining adds temporal parallelism (overlapping iterations in time). They can be applied together. In Q5.(c) the MAC loop is both unrolled by 4 (four parallel multipliers, DSP 2 → 4) and pipelined (II = 2), which cuts latency from 266 to 203 cycles. But neither technique rescues a poor structure: the combined 203/204 cycles still trails the simple fused loop at 134/135 cycles (Q4.(a)).

Question 6 – Memory Partitioning

Q6.(a) Array partitioning options applied to `shift_reg` in the Q5.(c) design (loops partitioned, MAC loop unrolled by 4):

Table 9: Array partitioning options (unrolling enabled)

<u>Partition of shift_reg</u>	<u>Latency (cycles)</u>	<u>Interval (cycles)</u>	<u>Loop II</u>	<u>Est. clock (ns)</u>	<u>Throughput</u>	<u>BRAM</u>	<u>DSP</u>	<u>FF</u>	<u>LUT</u>
No partitioning (Q5.(c))	203	204	2	7.206	0.68Mhz	6	4	999	813
block, factor 4	204	205	2	6.923	0.70Mhz	8	8	1,340	1,311
cyclic, factor 4	204	205	2	6.923	0.70Mhz	8	8	1,398	883
complete	201	202	2	6.923	0.72Mhz	0	8	5,295	7566

Complete partitioning gave the best performance, but only marginally: latency is 201 cycles, against 203 with no partitioning and 204 with block or cyclic partitioning. It moves `shift_reg` out of block RAM into registers (BRAM 6 → 0, FF 999 → 5,295) and doubles DSP usage (4 → 8) without improving the MAC loop II, which stayed at 2, so the extra parallelism bought nothing. Each iteration needs 2 cycles to compute $acc_{new} = acc_{old} + p0 + p1 + p2 + p3$, which also explains the 2 cycles per iteration seen in Q5.(c). Even the most aggressive hardware usage cannot bring the latency down any further with this loop structure.

Q6.(b) Effect of disabling either optimization:

Table 10: Loop unrolling and array partitioning, together and separately

<u>Design</u>	<u>Latency (cycles)</u>	<u>Interval (cycles)</u>	<u>Loop II</u>	<u>Est. clock (ns)</u>	<u>Throughput</u>	<u>BRAM</u>	<u>DSP</u>	<u>FF</u>	<u>LUT</u>
Unroll + partition (complete)	201	202	2	6.923	0.72MHZ	0	8	5,295	7566
Unroll only (Q5.(c))	203	204	2	7.206	0.68MHZ	6	4	999	813
Partition only (no unrolling)	267	268	1	6.912	0.54MHZ	8	2	595	605

Partitioning without unrolling gives no speed-up: latency is 267 cycles, the same as the partitioned loops in Q5.(b) (266 cycles), while BRAM rises from 3 to 8 and LUT from 395 to 605.

Unrolling without partitioning (Q5.(c)) reaches 203 cycles, and adding partitioning on top changes that by only a few cycles (201 to 204) while doubling DSP usage. In this design the speed-up comes from unrolling, and partitioning mainly adds area, because the MAC loop is limited by the accumulation and not by memory ports.

Question 7 – Best Design

Q7.(a) The highest-throughput design combines 5 optimizations (ap_int<5> for coef_t), loop partitioning into a shift loop (TDL) and a multiply-accumulate loop (MAC), full unrolling of both loops, complete array partitioning of shift_reg and c, and a function-level pipeline with II=1. Vitis HLS reports "Pipelining result: Target II = 1, Final II = 1, Depth = 31", so the design accepts a new input sample and produces an output sample on every clock cycle.

Table 11: Best-throughput FIR128 design

<u>Design</u>	<u>Latency (cycles)</u>	<u>Interval (cycles)</u>	<u>Loop II</u>	<u>Est. clock (ns)</u>	<u>Throughput</u>	<u>BRAM</u>	<u>DSP</u>	<u>FF</u>	<u>LUT</u>
Q7 best design	30	1	1	6.923	144.45 MHz	0	0	14,098	6,780
Baseline (Q3.(a))	135	136	1	6.912	1.06	2	2	588	315

Q7.(b) Resource usage of the best design against the baseline and the device:

Table 12: Resource usage of the best design compared with the baseline and with the xc7z020

	<u>BRAM 18K</u>	<u>DSP</u>	<u>FF</u>	<u>LUT</u>
Q7 best design (used)	0	0	14,098	6,780
Baseline (used)	2	2	588	315
Available on xc7z020clg400-1	280	220	106,400	53,200
Q7 utilization (%)	0	0	13	12

The baseline reuses a single multiplier and adder for all 128 taps and spends on the order of 128 clock cycles per sample. The best design gives every tap its own multiplier and adder and adds pipeline registers between stages, and it holds shift_reg in individual registers instead of a single memory so that all taps can be read in one cycle. The design trades area for throughput. Compared with the baseline (FF 588, LUT 315), the best design uses about 24 times the flip-flops and about 22 times the LUTs, while BRAM and DSP both drop from 2 to 0. FF is the resource closest to the device limit, at 13%.

Table 13: Grading requirements

<u>Requirement</u>	<u>Result</u>
Throughput of at least 5 MHz	144.45 MHz
Estimated clock period within 10 ns	6.923 ns
Resource usage within the Pynq-Z2 device	13%
Output matches out.gold.dat	Pass

Programs Summary and Repository Structure:

The repository contains a top-level `fir` folder with `Report.pdf` and one folder per design. Each folder holds only `fir.h`, `fir.cpp`, `fir_test.cpp`, `Makefile`, `__hls_config__.ini`, `fir_test.log`, `fir_csynth.rpt` and `fir.tcl`.

Table 14: Repository folders and the design each one contains

<u>Folder</u>	<u>Design</u>	<u>Reported in</u>
Q1	FIR11, functionally correct, no pragmas	Q1
Baseline	FIR128, no optimizations or pragmas	Q3.(a)
Q2	FIR128 with minimum <code>coef_t</code> and <code>acc_t</code> bitwidths	Q2.(b)
Q3	FIR128 with pipelining turned off	Q3.(b)
Q4	FIR128, automatically pipelined, conditional removed	Q4.(a)
Q5&6	FIR128 with loop partitioning, unrolling and array partitioning	Q6.(a)
Q7	FIR128 with the highest throughput	Q7